Gravisonde Architecture

Diagram: camera → EC2 Graviton4 (COOL AMI) → OpenCV 5 pipeline → CloudWatch + S3 → Next.js 16 dashboard (SSE) → Judge's browser

---

flowchart BP %% External subgraph Judge["Judge's Browser"] direction TB DSS["Dashboard\n(App Router, SSE)"] end %% AWS subgraph AWS["AWS Cloud"] direction TB VPC["VPC (2 AZs, 0 NAT)"] EC2["EC2 c8g.xlarge\n(Graviton4)\nCOOL AMI + OpenCV 5"] S3["S3 Bucket\nbenchmark artifacts"] CWLog["CloudWatch Logs\n/p/aws/gravisonde/pipeline"] CWMetric["CloudWatch Custom Metrics\nFPS, latency p50/p95/p99"] ALB["Application Load Balancer\nHTTPS:443, HTTP:80"] end %% Connections Camera -->|RTSP / File| EC2 EC2 -->|fps, latency_ms, detections| CWMetric EC2 -->|CSV logs| S3 EC2 -->|logs| CWLog EC2 -->|metrics| CWMetric ALB -->|HTTPS| DSS DSS -->|SSE metrics| Judge DSS -->|Live video| Judge DSS -->|Benchmark Download| Judge

---

Layer Summary

Layer	Tech	Notes
Camera	RTSP / file source	Sample video at `/data/sample.mp4`
Edge	EC2 c8g.xlarge (Graviton4)	$0.136/hr, COOL AMI with Arm KleidiCV
Compute	OpenCV 5 DNN + Features2D	YOLO11n ONNX + ALIKED + LightGlue
Telemetry	CloudWatch + S3	Per-frame CSV upload + custom metrics
App Layer	Next.js 16 (App Router)	React 19, Tailwind v4, SSE streaming
UI	LiveVideoCanvas + MetricsCharts	Real-time fps/latency cards
Ingress	ALB (HTTPS:443, HTTP:80)	Public endpoint for judges